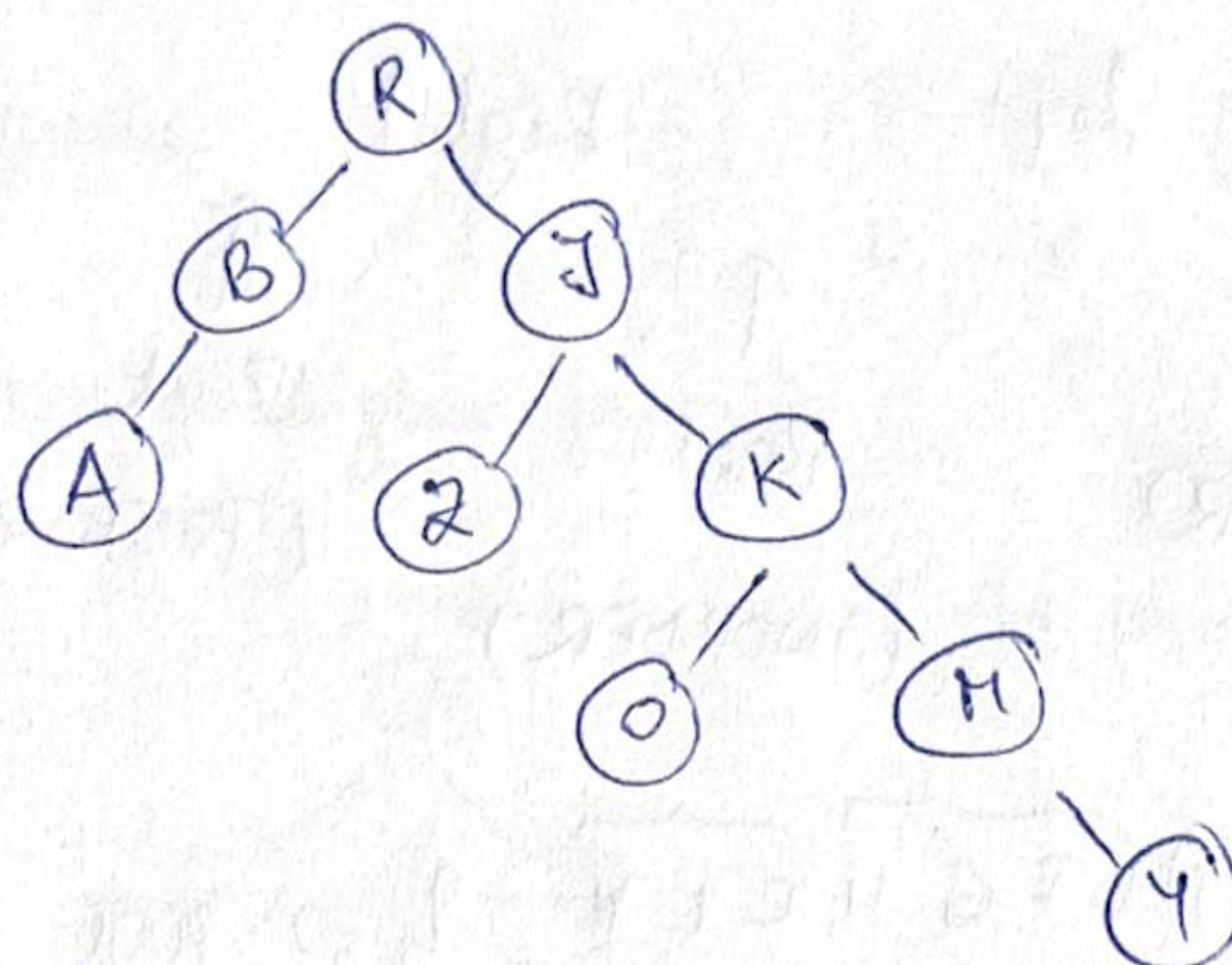


Binary Tree Traversal



1) Preorder: root-left-right

preorder(root, left, right): R B A J Z K O M Y

inorder(left, root, right): A B R Z J O K M Y

postorder(left, right, root): A B Z O Y M K J R

in algorithm $\rightarrow$ @ visit [node].info $\rightarrow$ at the end $\Rightarrow$ postorder
 $\hookrightarrow$...
 $\hookrightarrow$...

root-right-left = reverse of postorder?

2) - postorder traversal with one stack. -

$\hookrightarrow$ know when to visit each node based on how many times we that encountered that node. $\Rightarrow$ pop it for the second time $\Rightarrow$ visit node.

Stack: ~~R~~ ~~B~~ ~~A~~ ~~R~~ ~~K~~ ~~Z~~ ~~H~~ ~~K~~ ~~O~~ ~~K~~ ~~M~~ ~~M~~ ~~Y~~

cn: ~~R~~ ~~B~~ ~~A~~ nil ~~A~~ nil ~~B~~ nil ~~R~~ ~~Z~~ nil ~~Z~~ nil ~~J~~ ~~O~~ nil ~~K~~ ~~M~~ nil ~~M~~ nil ~~Y~~ nil ~~Y~~ nil
~~M~~ nil ~~K~~ nil ~~Z~~ nil R

visited: A B Z O Y M K J R

push right child, root

current node = nil $\Rightarrow$ pop

force to pop another elem from stack swap them (ex: M, Y)

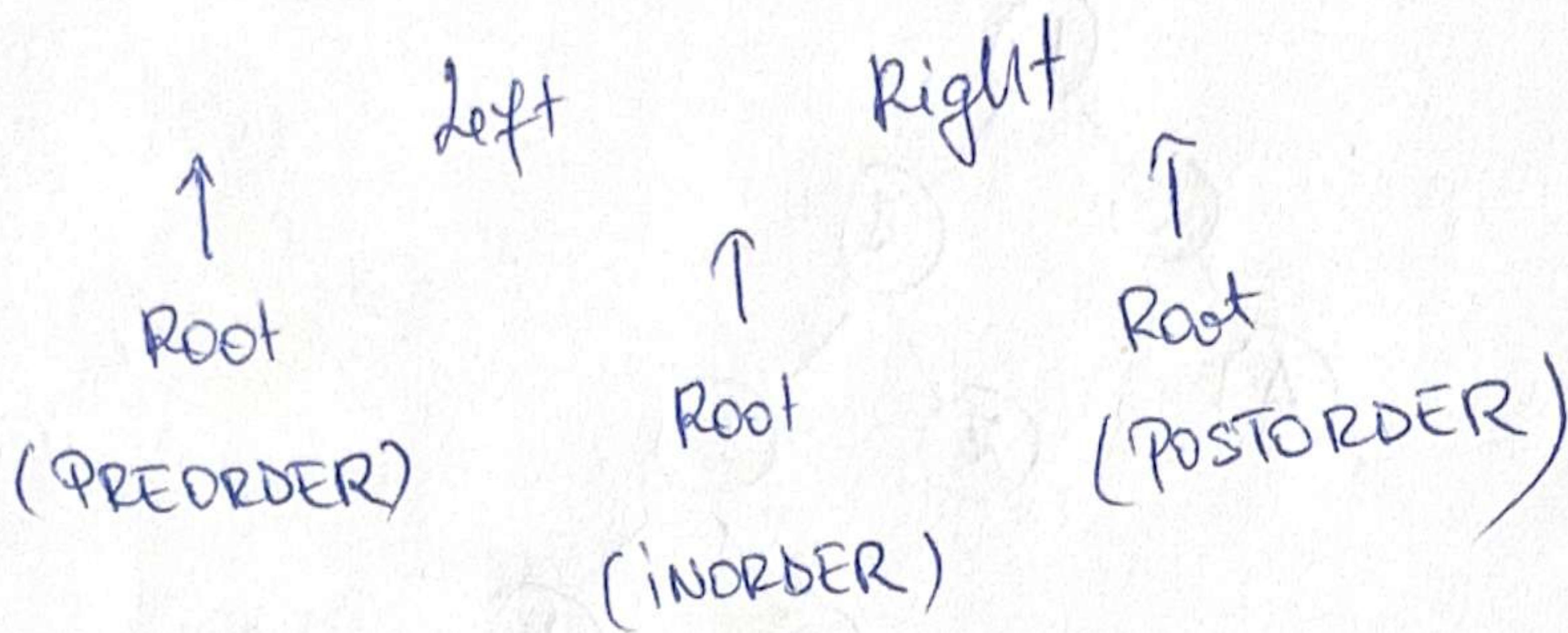
$\Rightarrow$ top of stack is exactly that child $\Rightarrow$ we are not allowed to...

$\hookrightarrow$ has a right child $\Rightarrow$ skip $\Rightarrow$ pop another

$\hookrightarrow$ $\nexists$ a right child $\Rightarrow$ push

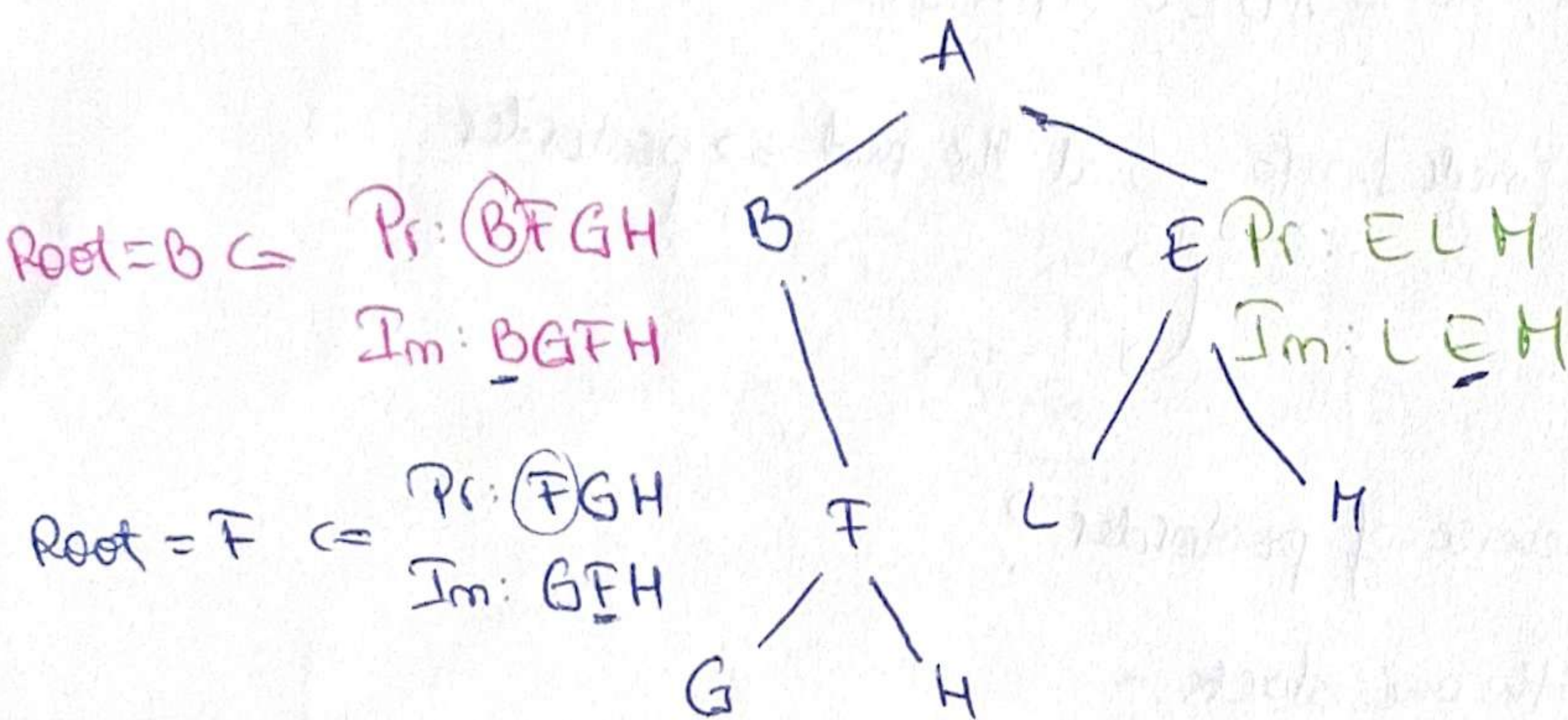
3) level-order-traversal (

Left is always before Right



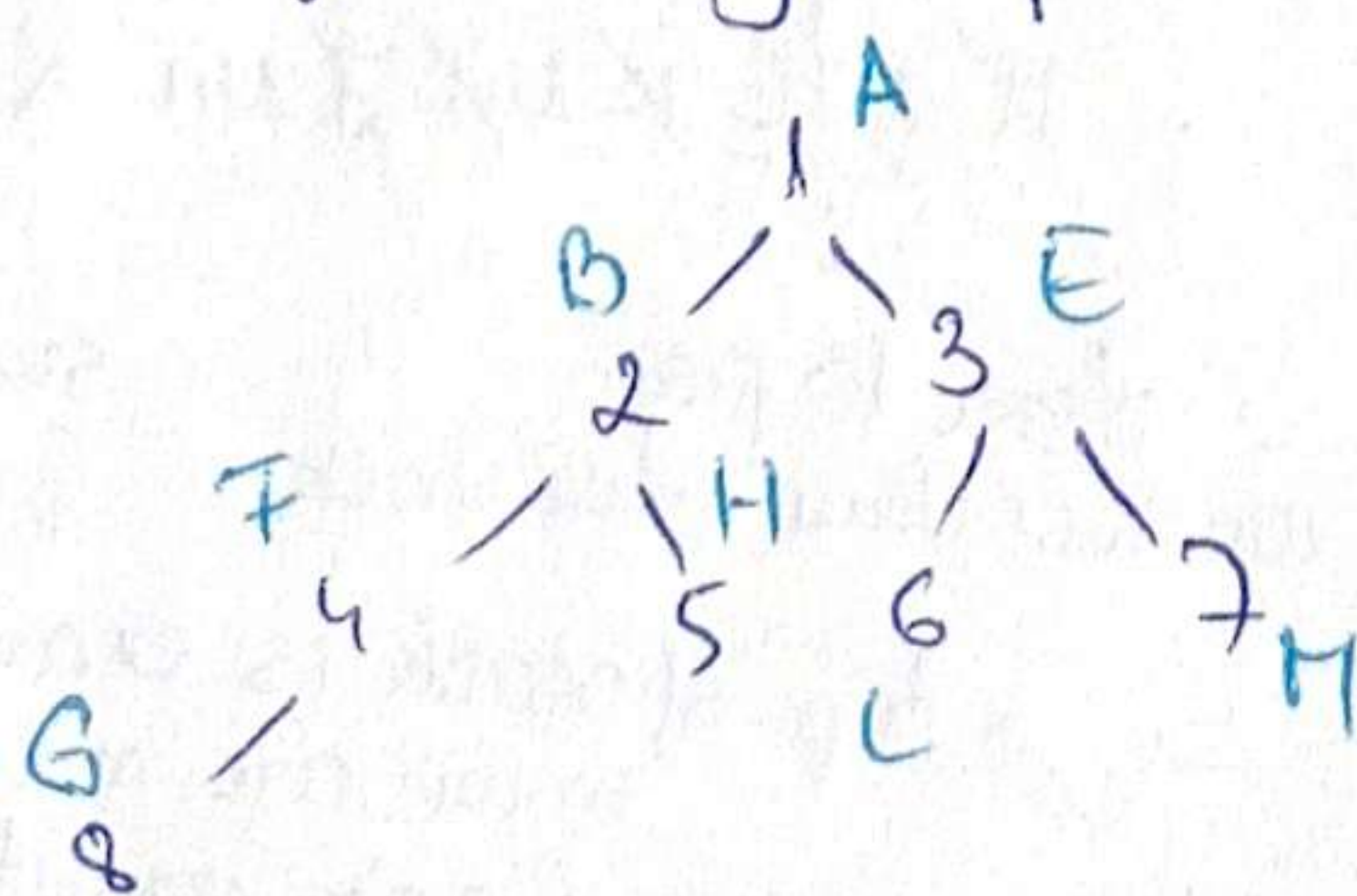
4) I know preorder: A B F G H E L M $\Rightarrow$ ROOT = A
 inorder: B G F H A L E M

I don't know the tree



! preorder + postorder $\Rightarrow$ we can't do this

5) How to start if binary heap structure for preorder: A B F H E L M



(everything on the left & everything on the right?)

6) Huffman coding = assigning different length messages to each letter in a

A - 00

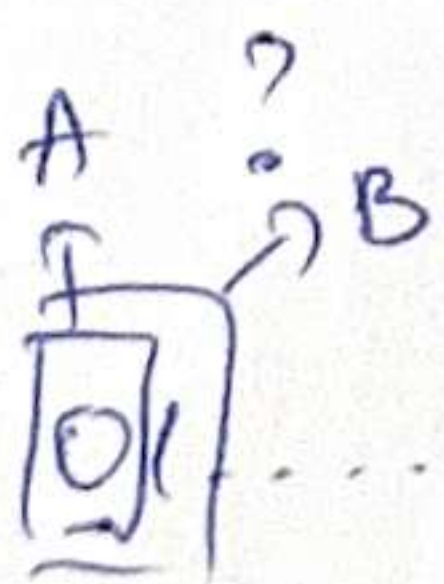
B - 01

C - 10

D - 11

I get message: 00/00/01/10/00/10/10/
 A A B D...

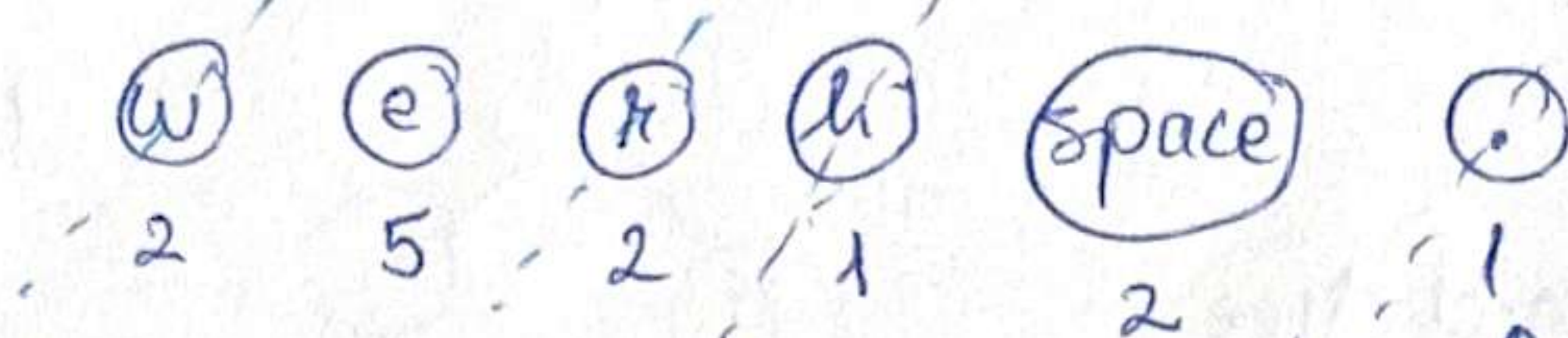
way that shorter codes are assigned to more frequent letters, longer codes for less frequent ones
 $\Rightarrow$ problem: the size? no code can be the prefix of another code



message: WE WERE HERE.

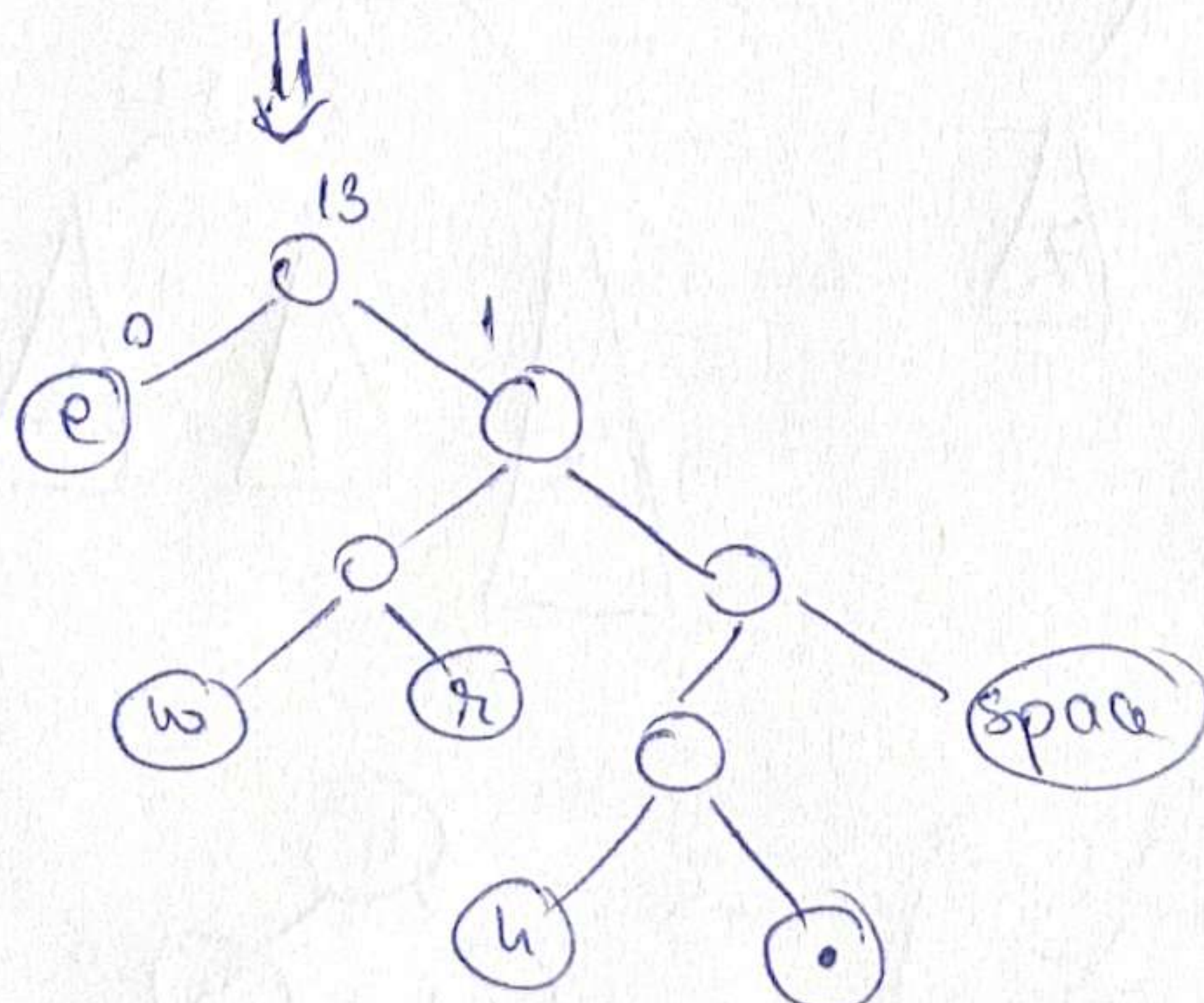
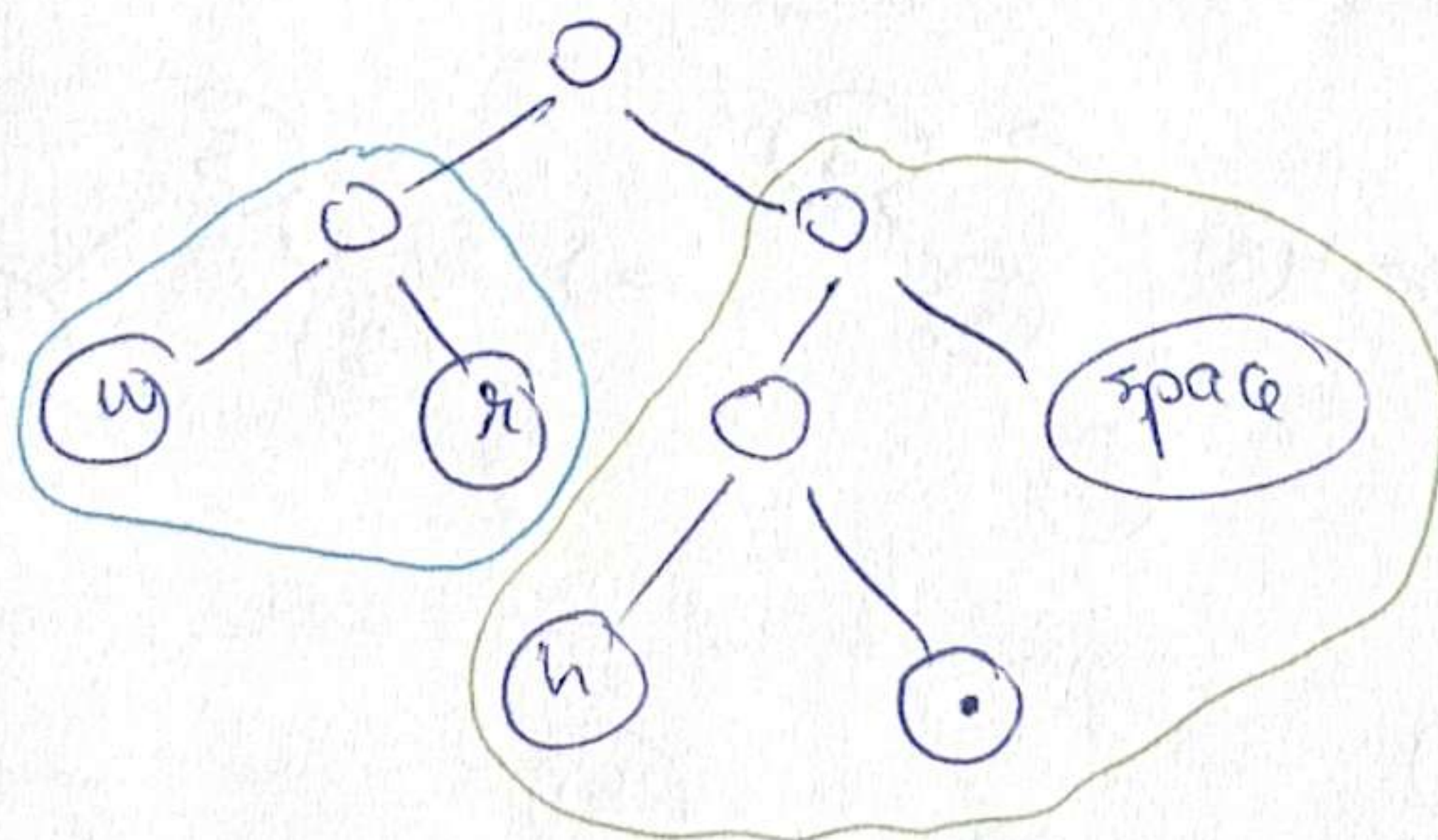
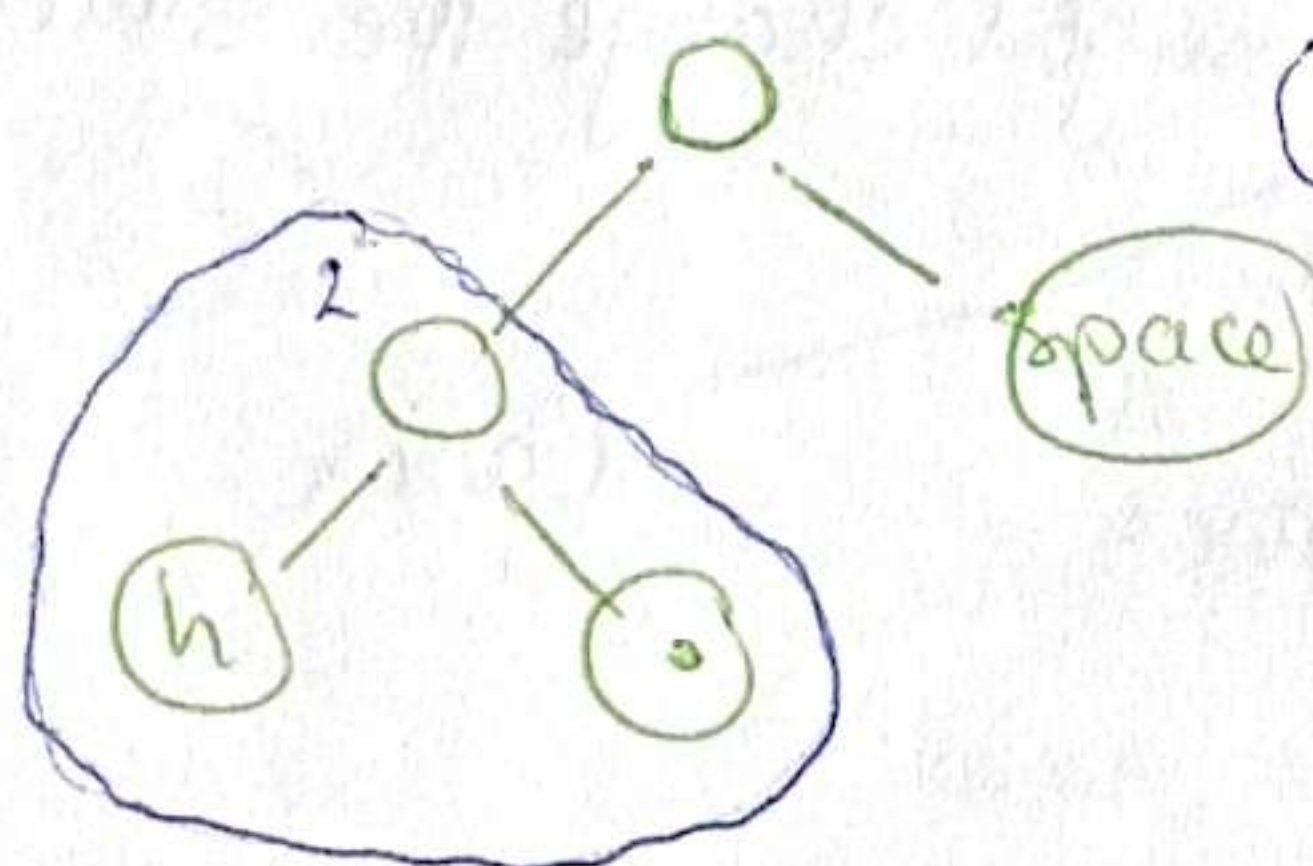
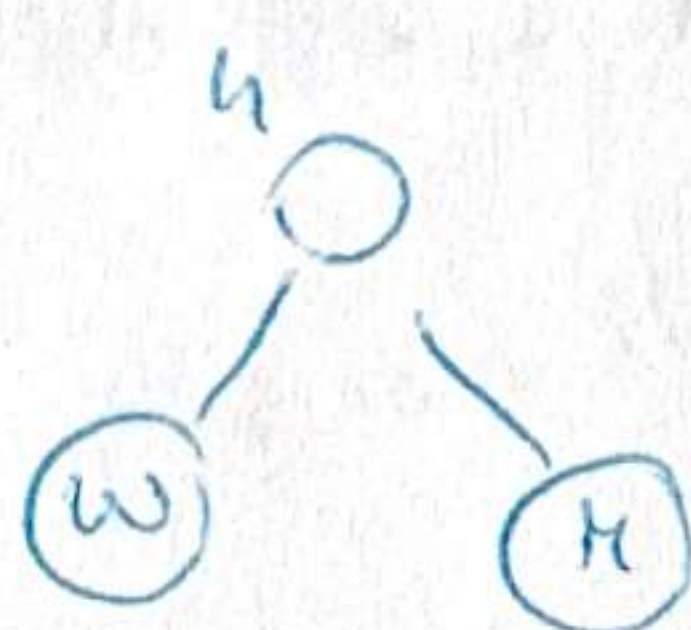
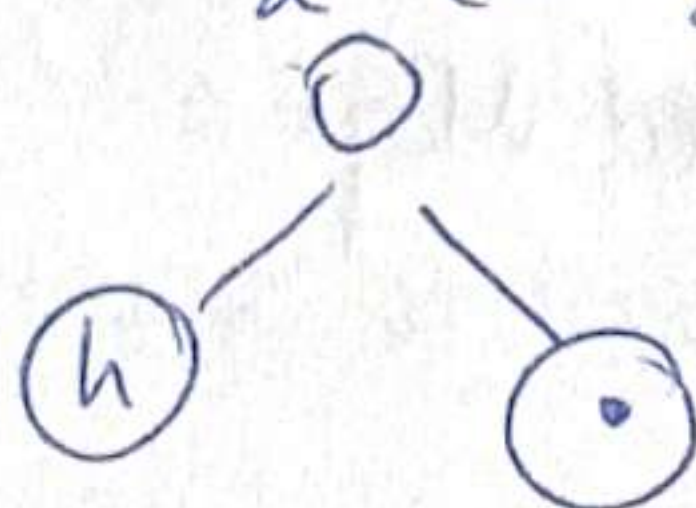
→ frequency of each letter

→ priority queue



smaller number $\Rightarrow$ higher priority.

2 $\leftarrow$ sum of the priorities of the children



Why separate? (still page 2)

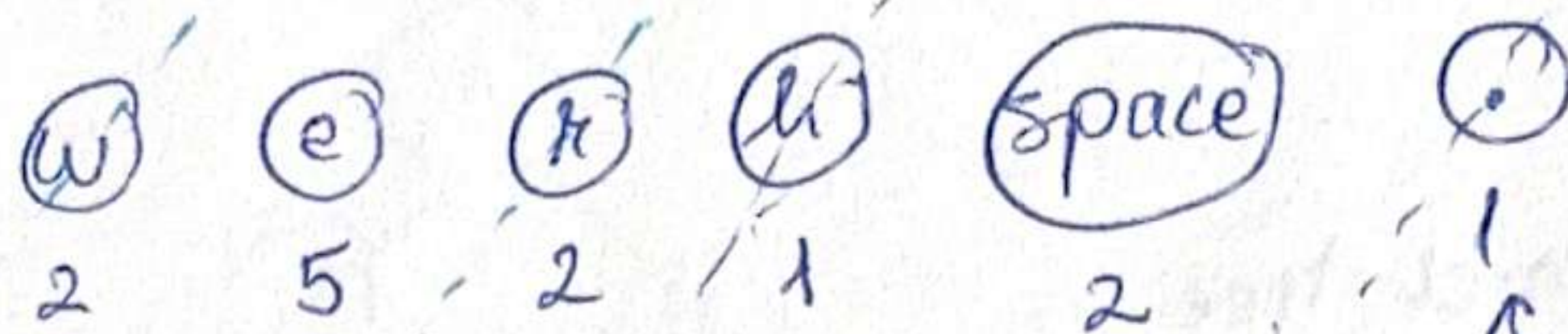
[Handwritten signatures and scribbles]

$e: 0$
 $w: 100$
 $r: 101$
 $h: 1100$
 $space: 1101$
 $sp: 111.$

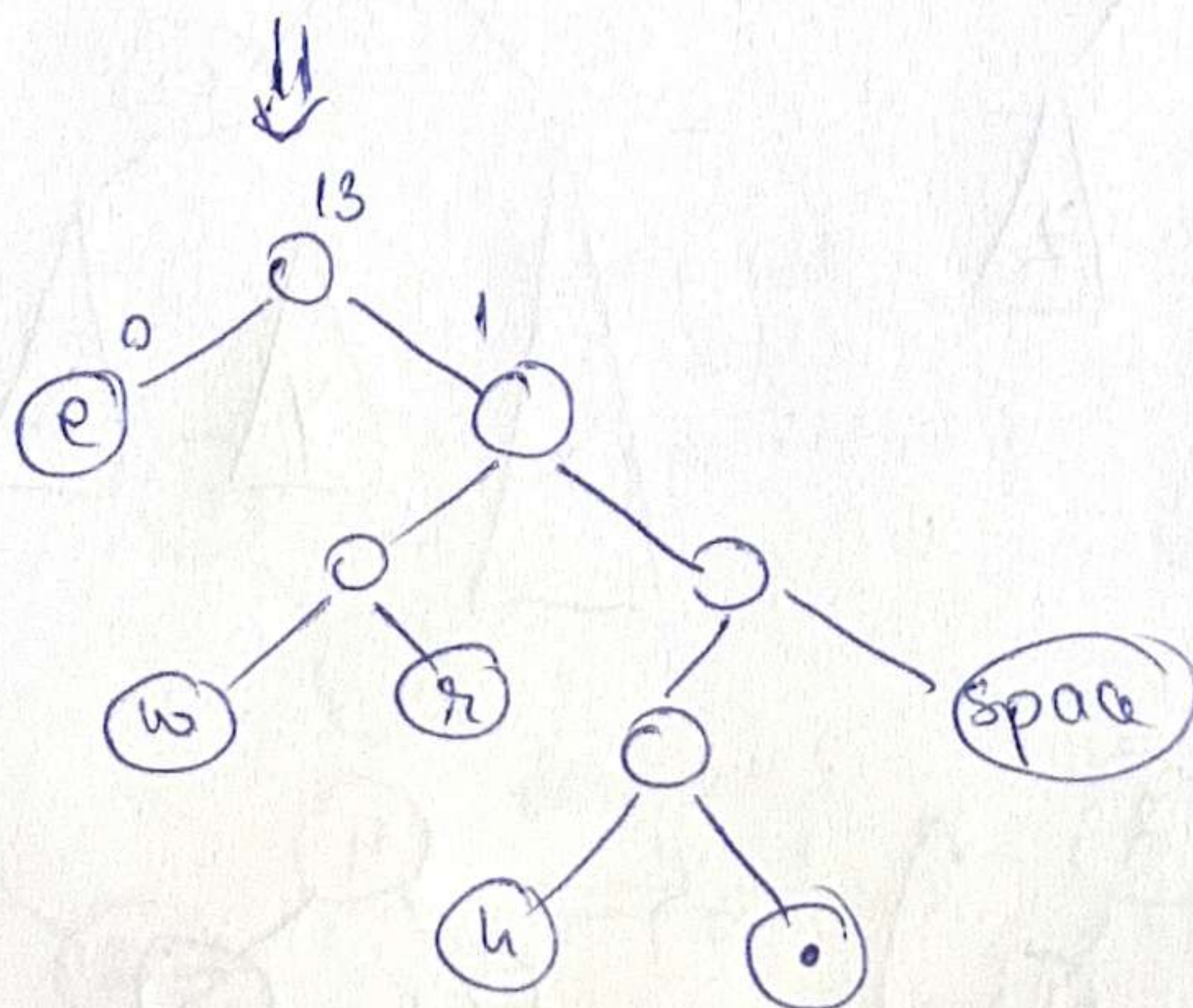
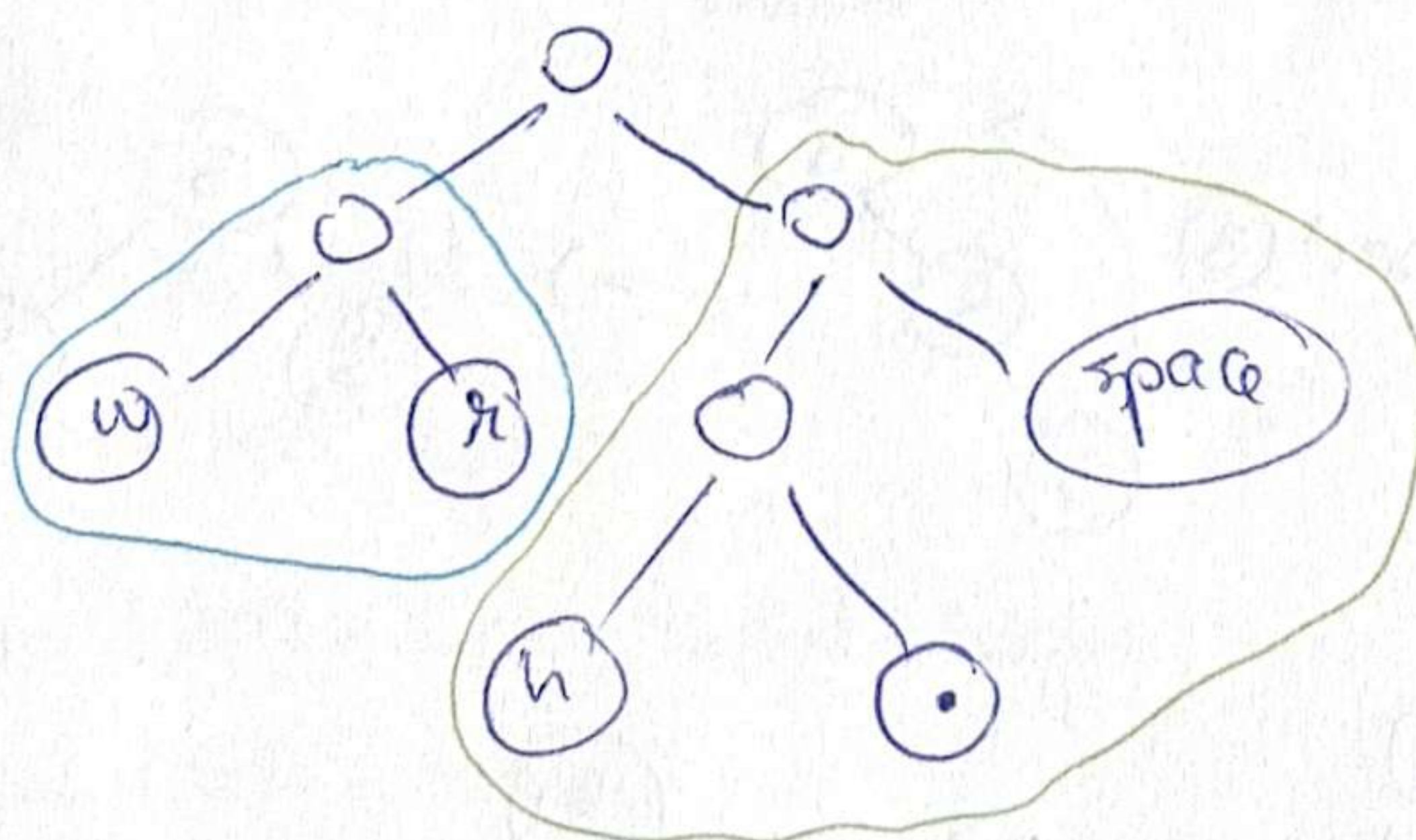
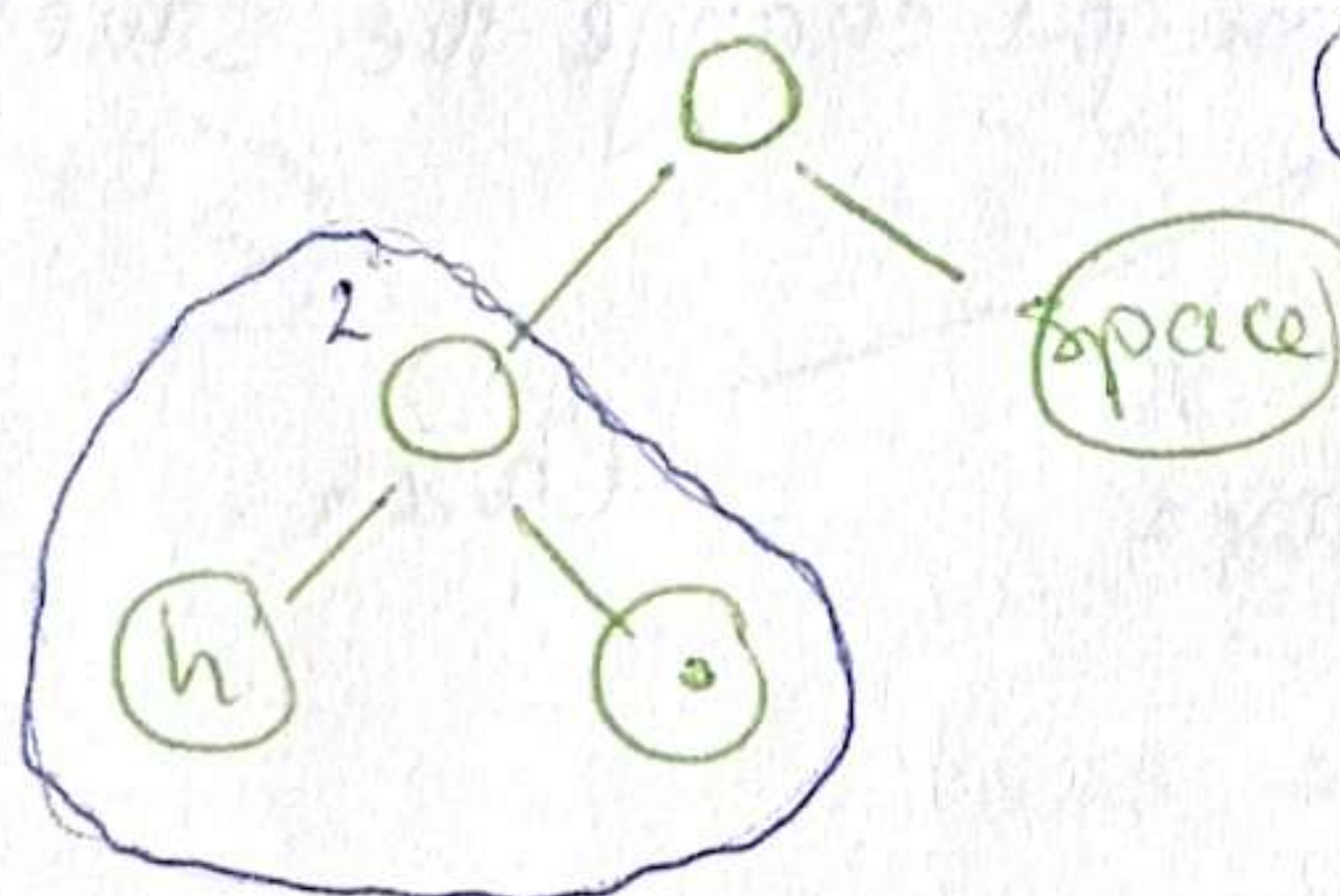
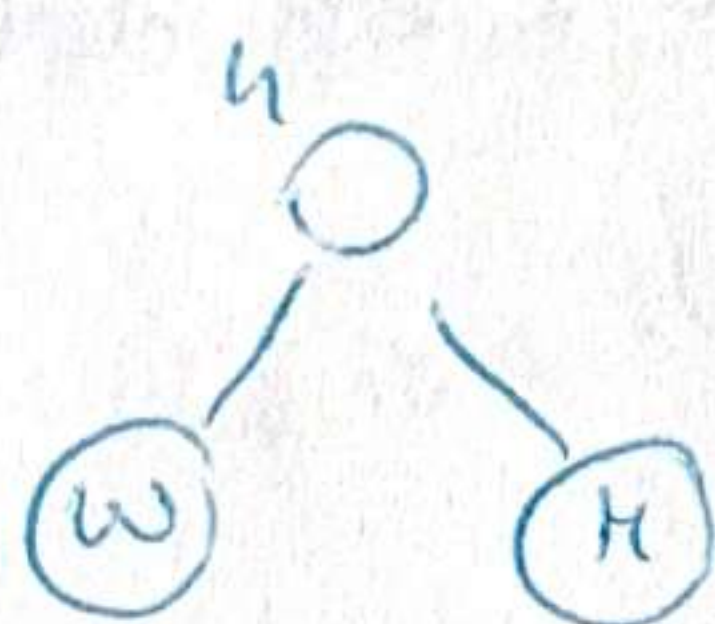
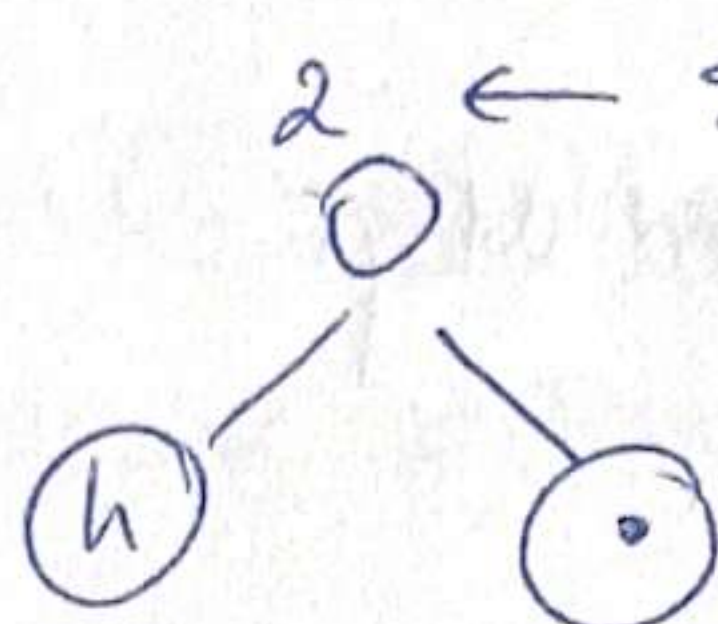
message: WE WERE HERE.

→ frequency of each letter

→ priority queue



smaller number $\Rightarrow$ higher priority.



Why separate? (still page 2)

[Handwritten scribbles]

$e: 0$
 $w: 100$
 $r: 101$
 $h: 1100$
 $.: 1101$
 $sp: 111.$

Decode: 10110000101001
 r w e e

"left" $\rightarrow$ "right"
 0 $\rightarrow$ go left
 1 $\rightarrow$ go right

7) Binary Search tree

(nodes on the left $\leq$ nodes on the right)

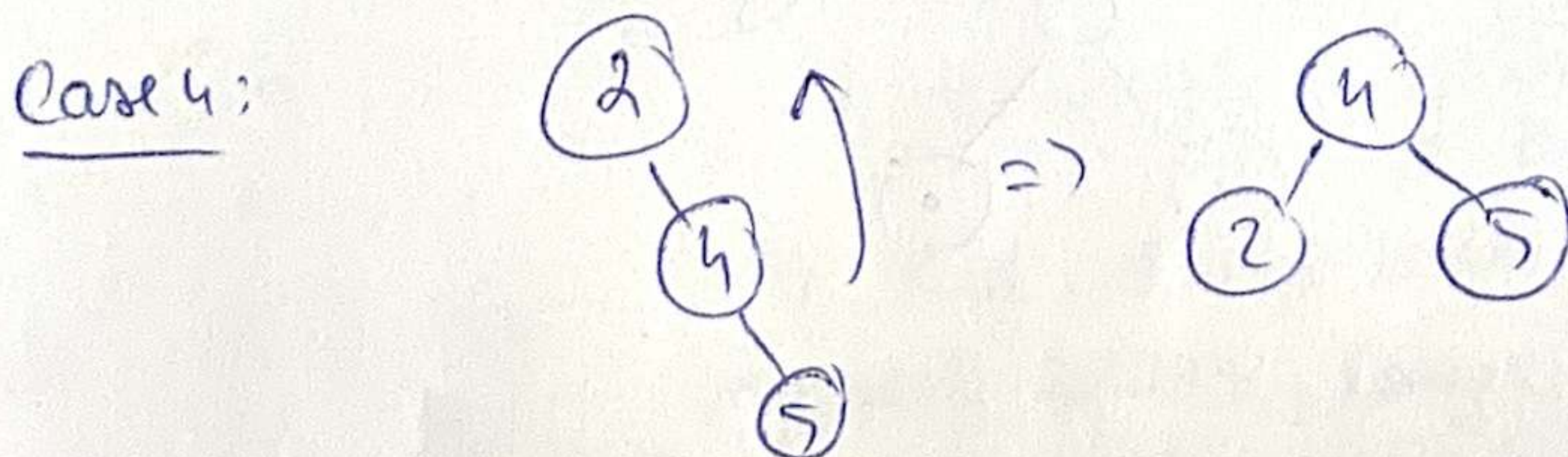
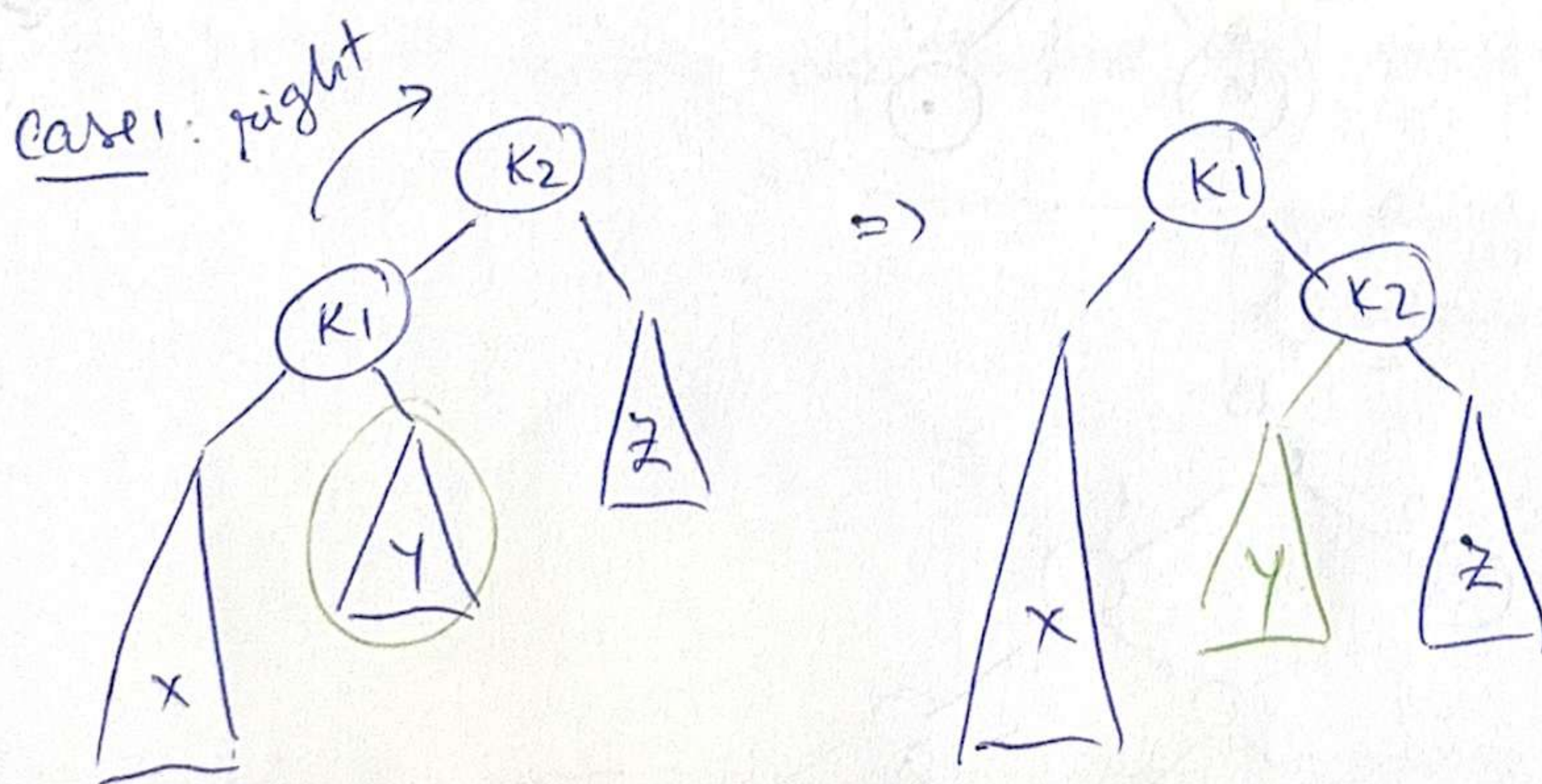
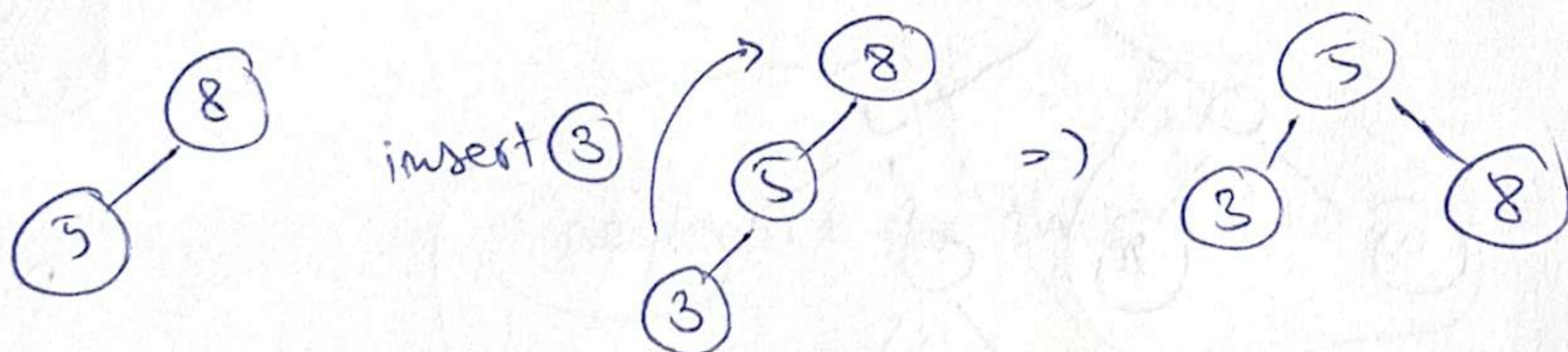
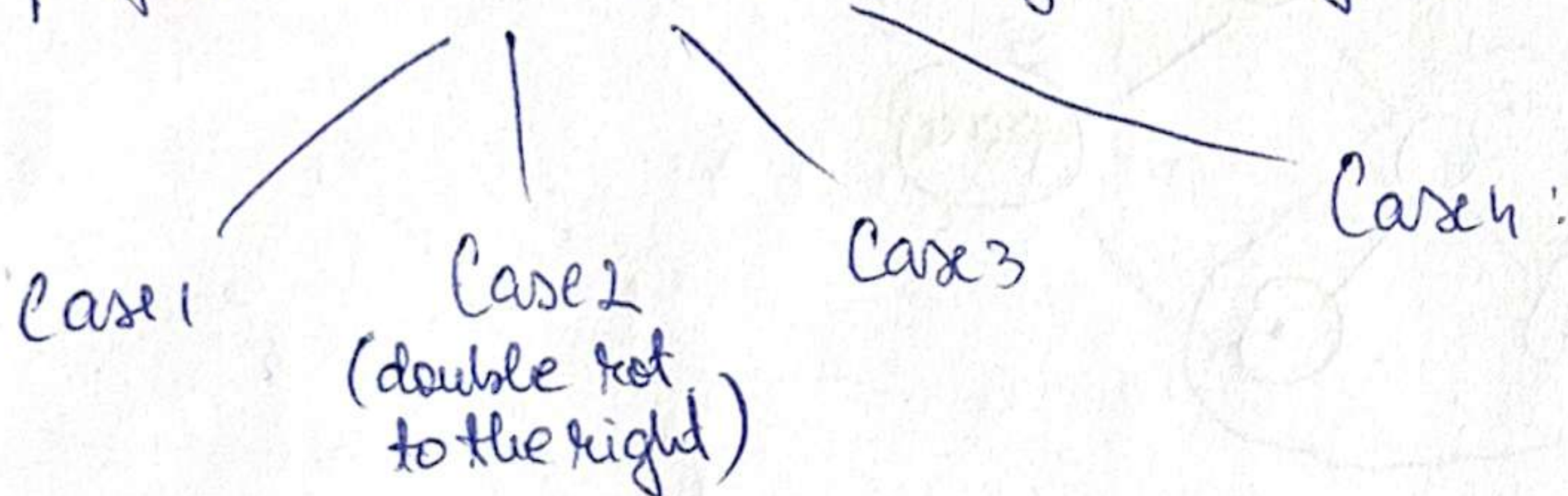
- $\rightarrow$ sorted dynamic array
- $\rightarrow$ sorted DLL
- $\rightarrow$ BST

} Sorted Map

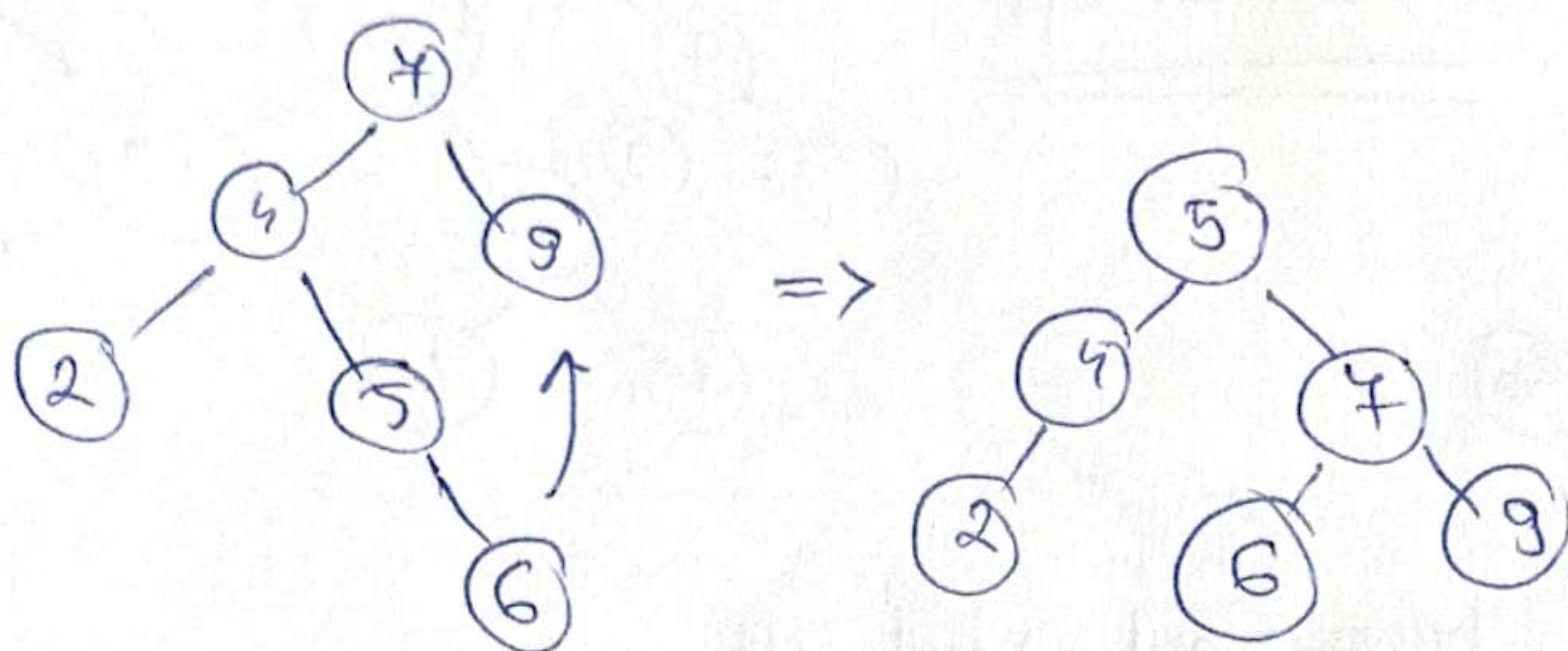
$\hookrightarrow$ improvement: balanced BST : left-child - right-child $\leq \log_2 m$
 $\Downarrow$
 logarithmic complexity.

8) AVL trees

$\rightarrow$ perform rotations (we might change the structure)

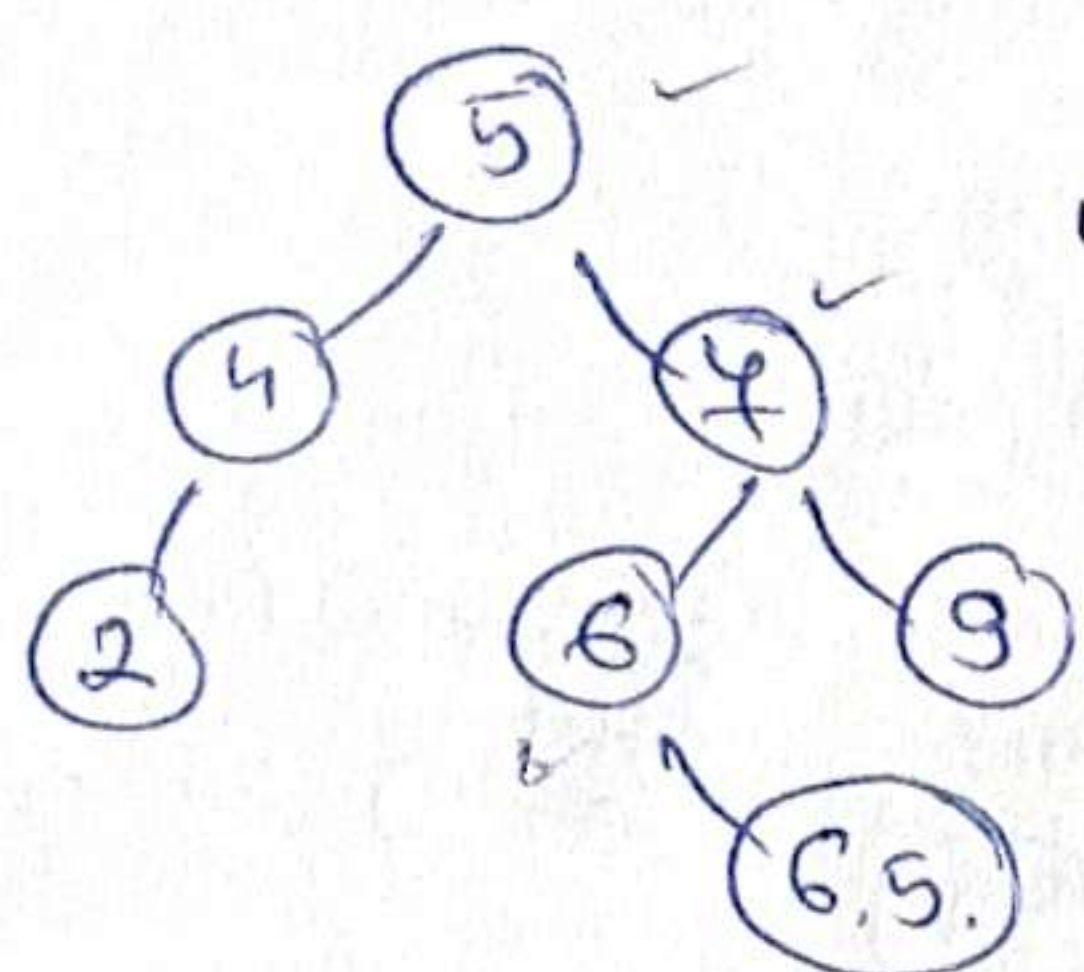


insert (6) => (7) would be the imbalanced node.

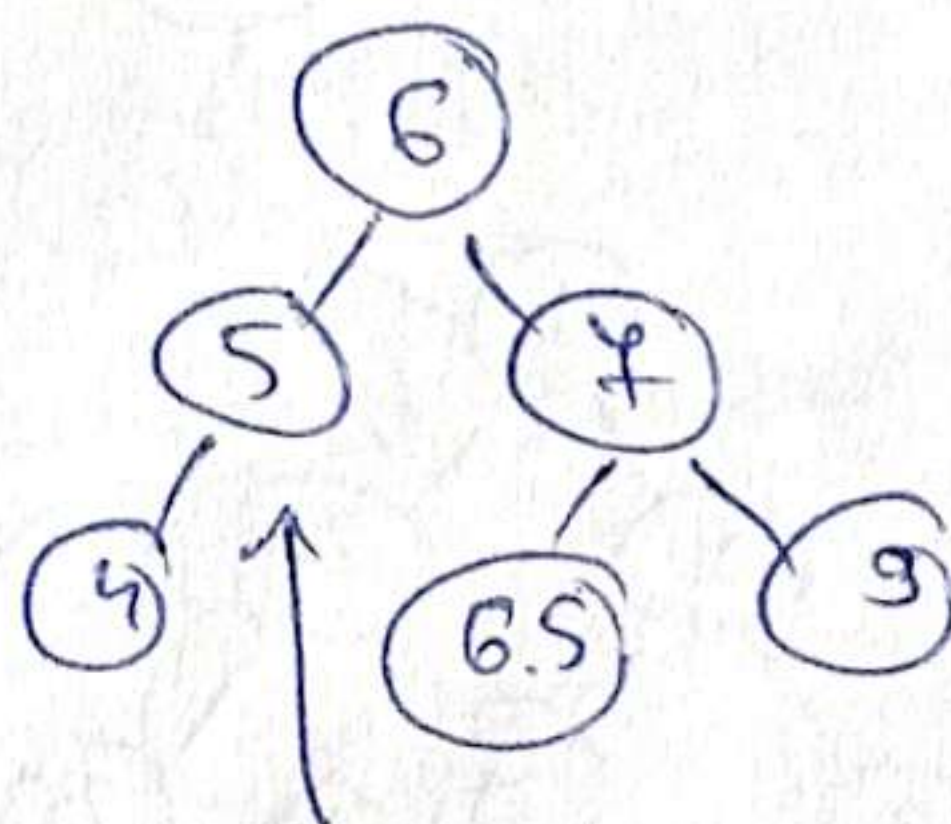


put (6) to the right
(because 2 does not have children)

remove 2 and add 6.5 instead



why is 6 the new root?



why (6.5) not here? because it's a BST
and we would have (6.5) on the left of (6)
which is not allowed in a BST

9) rot. left or right?

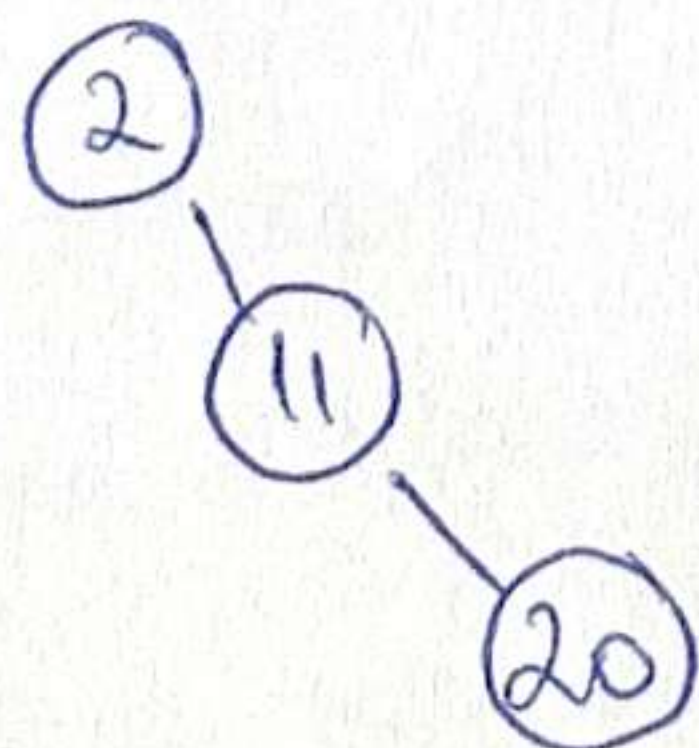
"heavy left" => rot. right.

"heavy right" => rot. left.

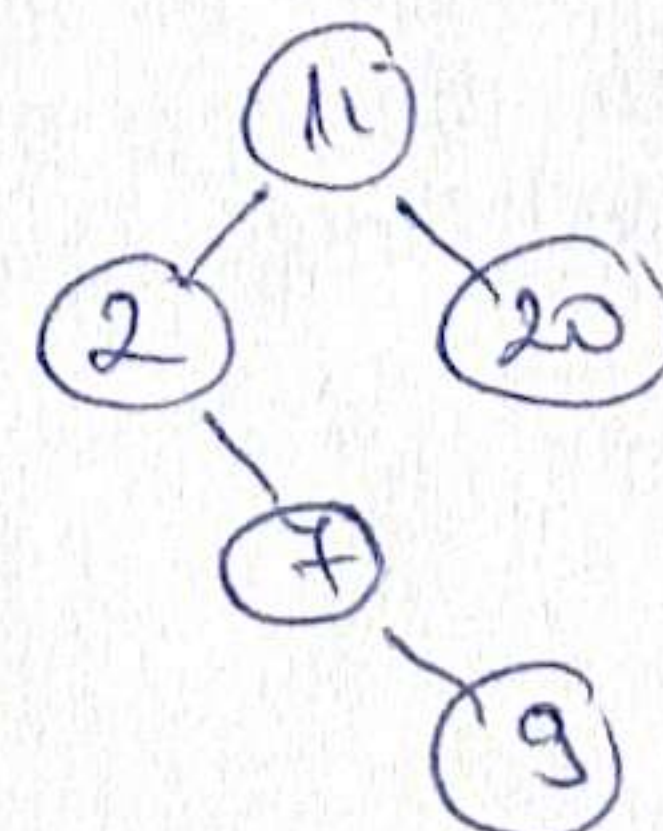
simple rotation (left + left, right + right)

double rotation (left + right)

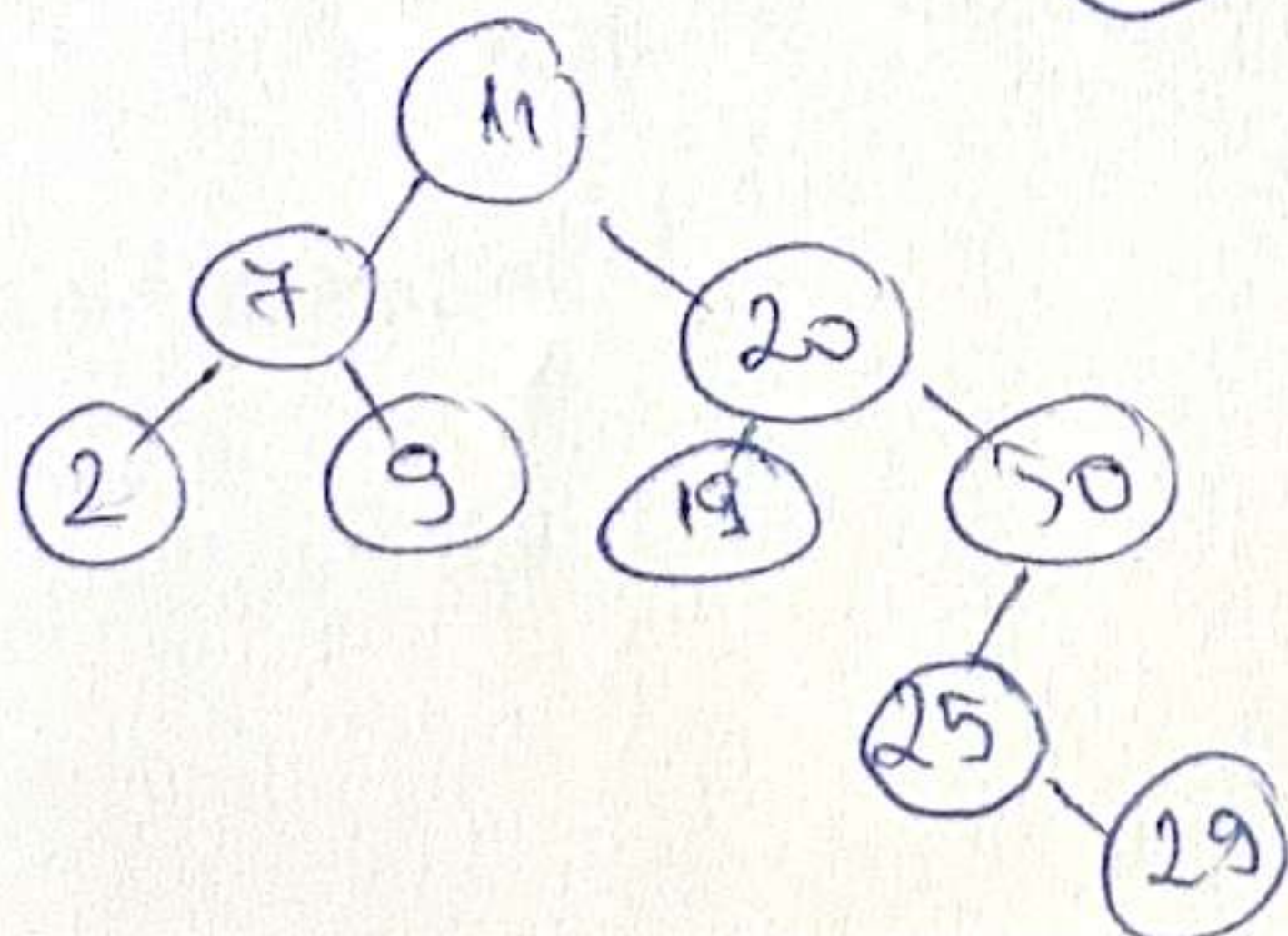
10) empty AVL tree:



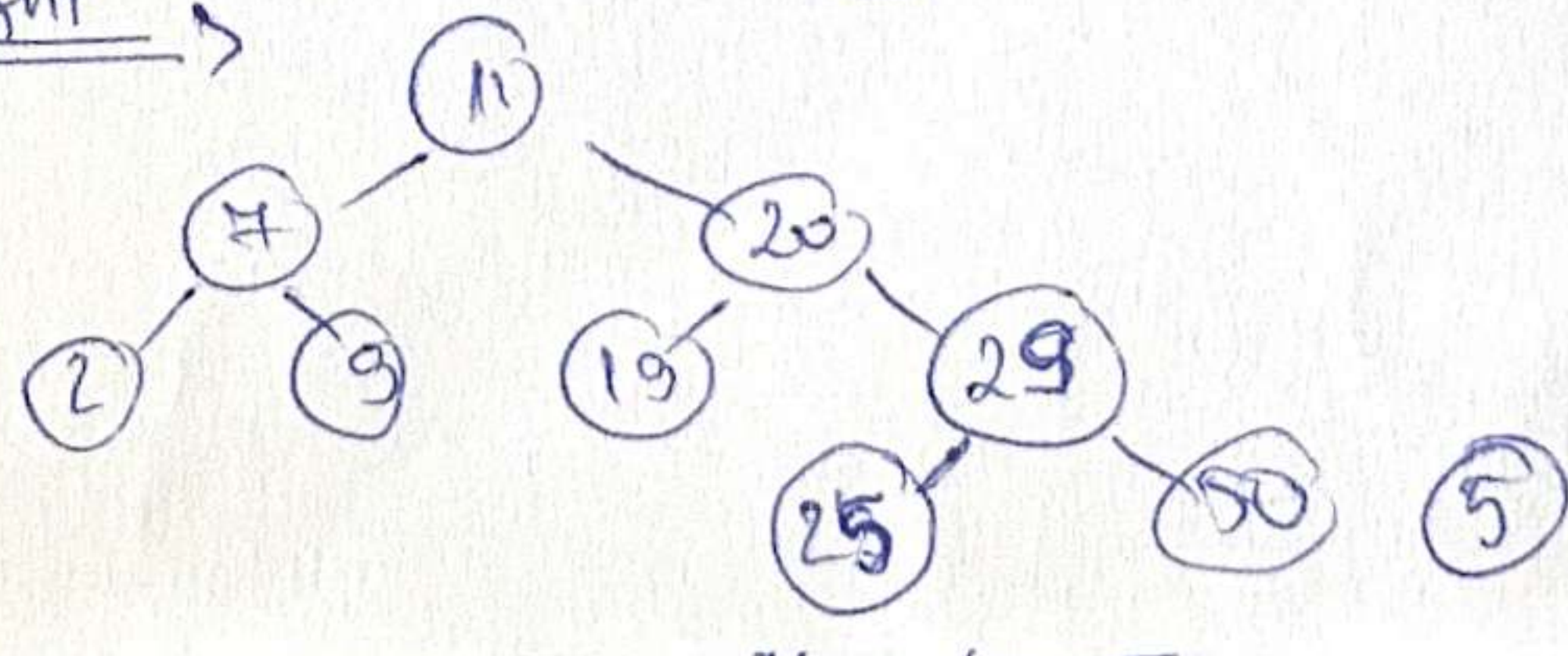
single rot.
left



single rot.
left +

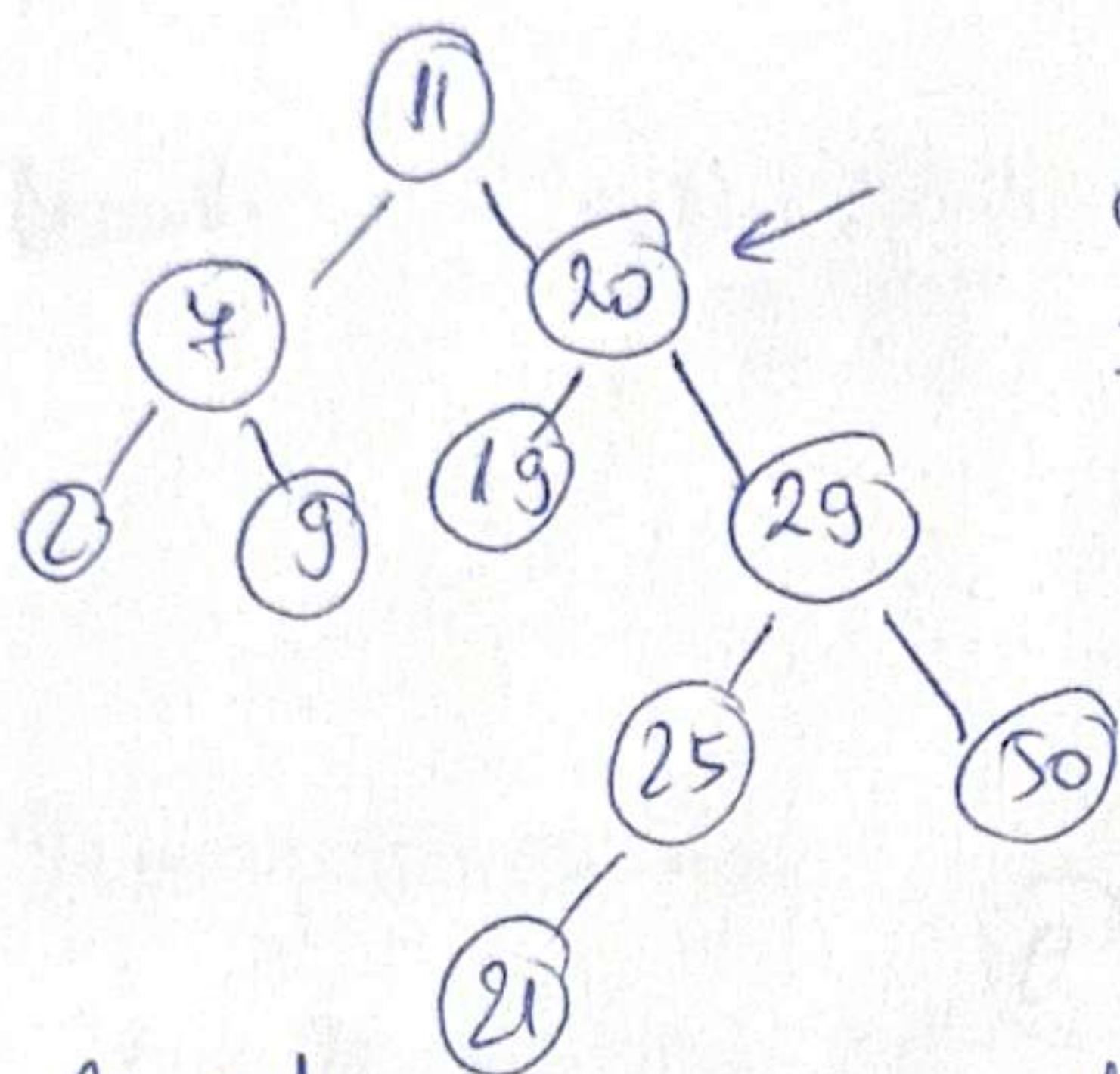


double rot.
right

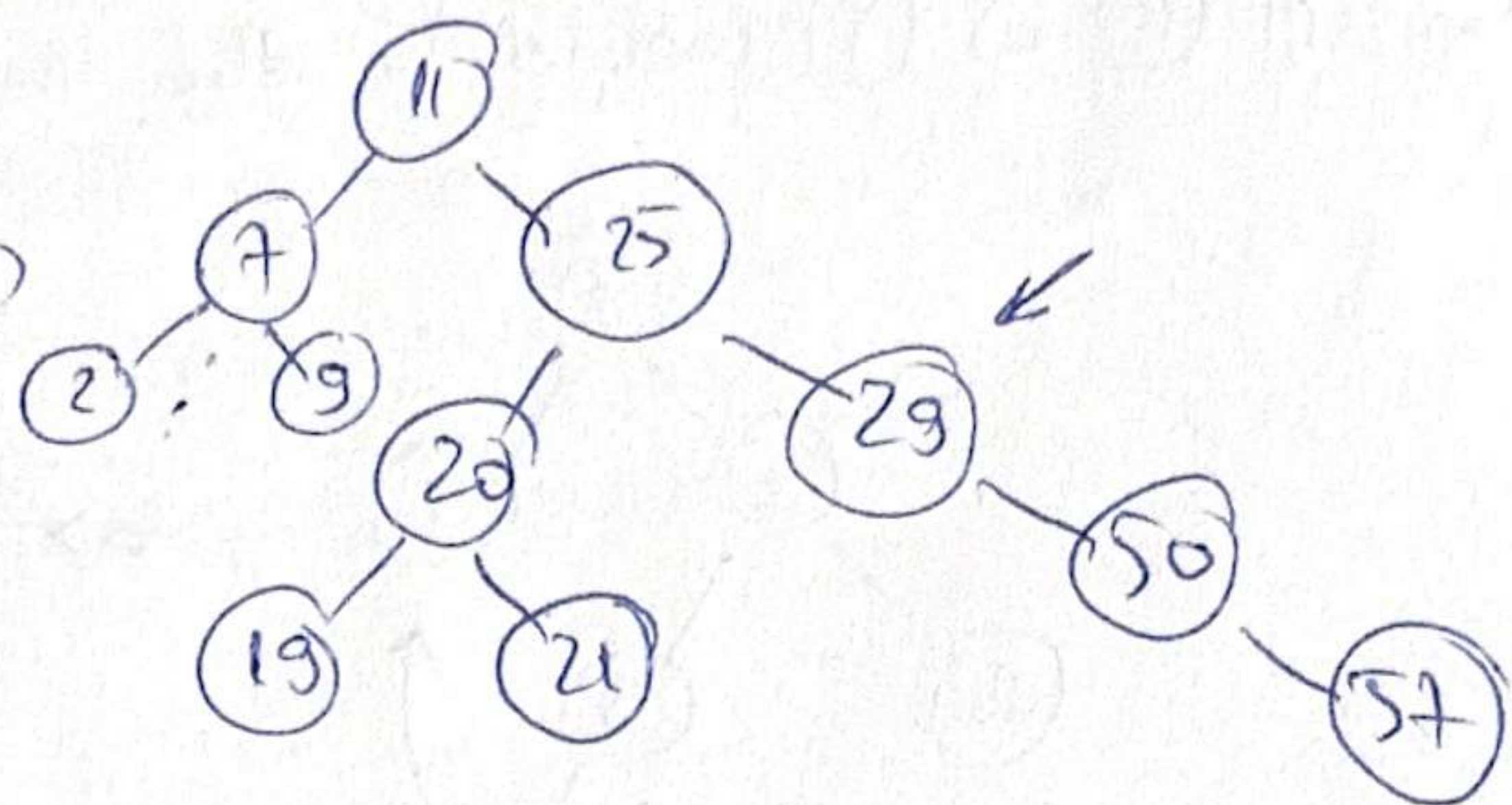


the LEFT

why not 25
right of 19?

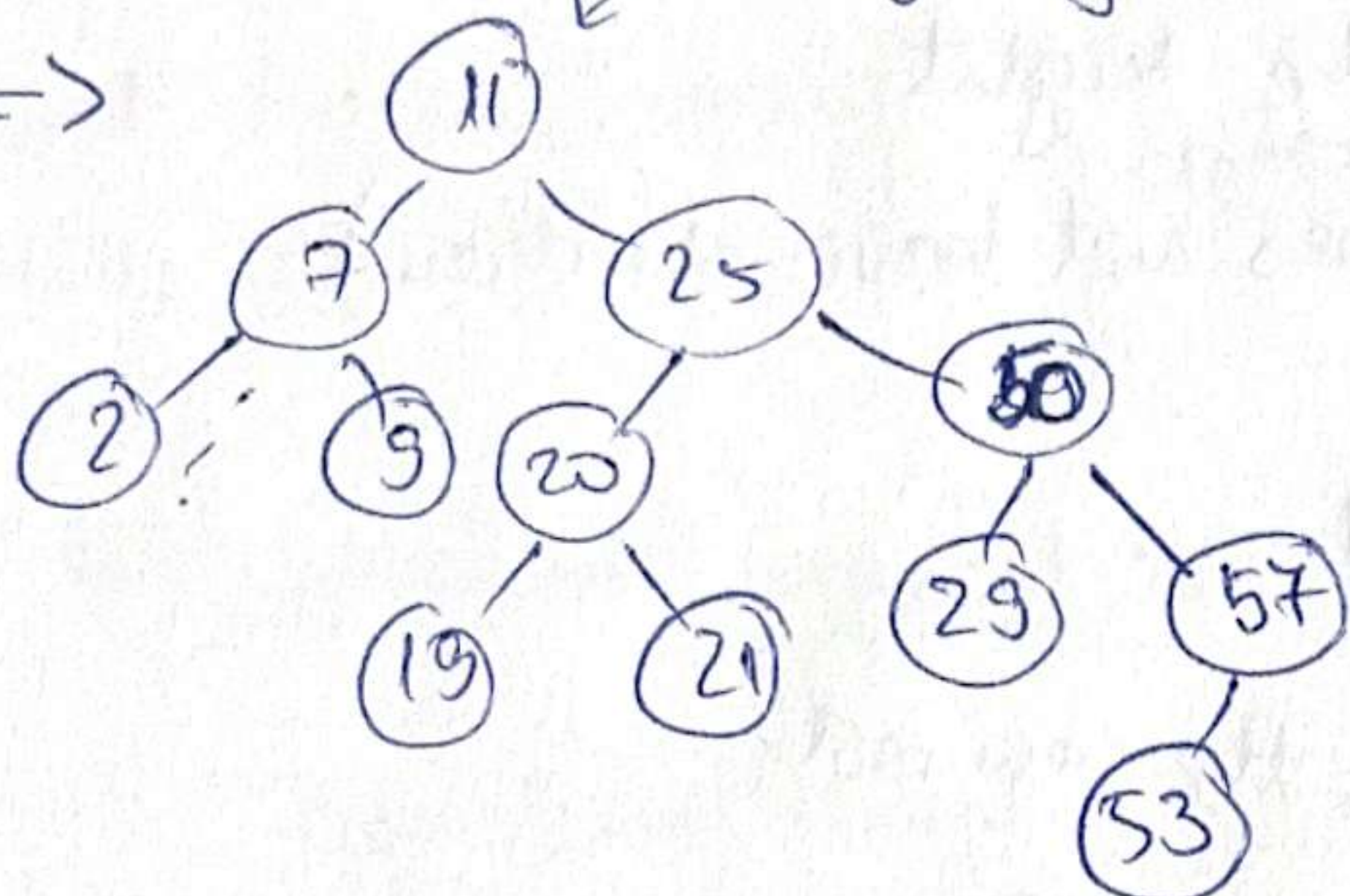


double rot. left

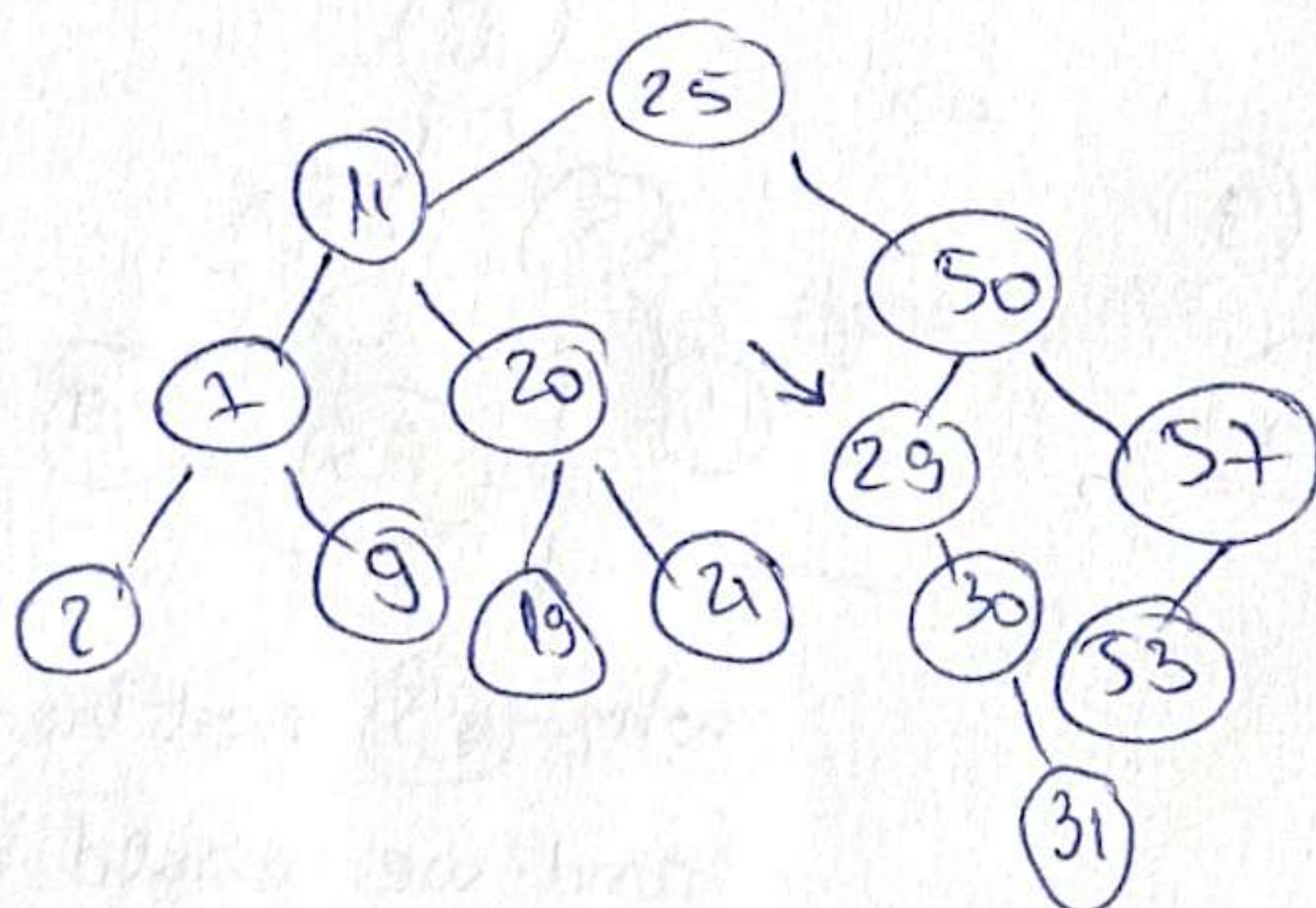


single rot. left.

heavy-right => rot. left.



single rot. left.



single rot. right.